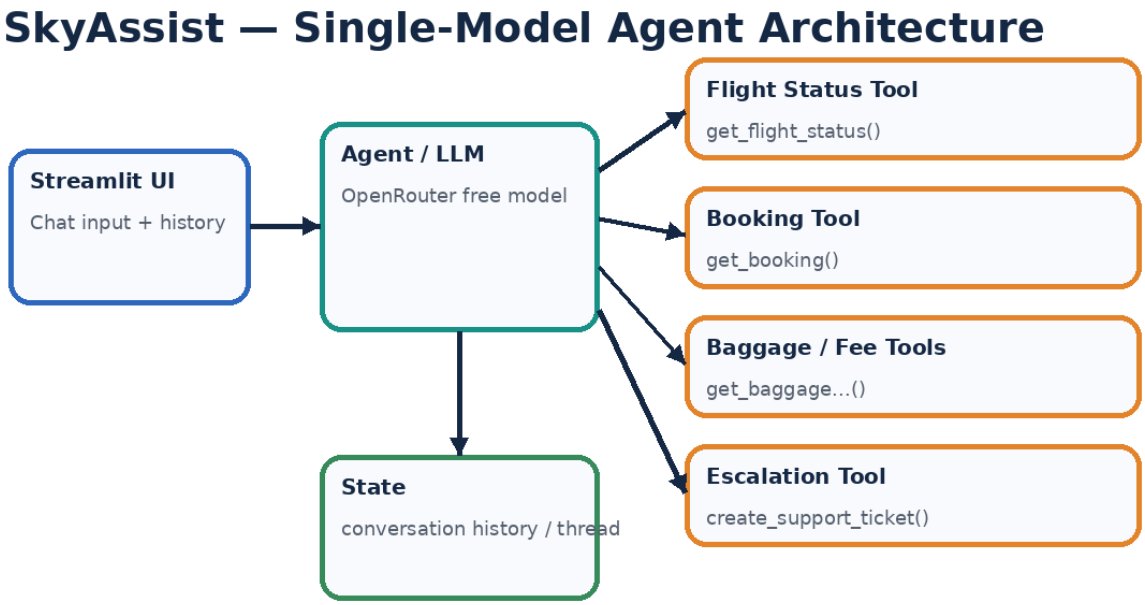
Project

Design & Implement AI-Driven Airline Customer Support Agent

Jetking

SkyAssist

Streamlit UI • OpenRouter Free Models • Function Calling • Stateful Conversation



PROJECT FORMAT 7 sessions / 28 hours • Student-Centric Self Paced • Synthetic airline data • Single-model agent

Disclaimer: *This is a fictional sample project. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.*

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking

Table of Contents

Section	Content
1	Overview
2	Theory
3	Schematic Diagram
4	Pre-Lab Setup
5	Project Steps
6	Quick Reference
7	Output
8	Evaluation Criteria
9	Reference Links

Project Note to Students and Academic Team

- Recommended team size: 4 students, or individual work if required by the academic delivery.
- Students should perform the steps, record observations, keep screenshots of important checkpoints and demonstrate the final application.
- Use only synthetic airline data. Do not connect to real airline booking, payment, identity or operational systems.
- Recommended path: Streamlit + OpenRouter + Python functions + current LangChain agent runtime. CrewAI is an optional alternative.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

1. Overview

Aim

Design, implement and demonstrate SkyAssist, a single-model airline customer-support agent with a conversational Streamlit UI, OpenRouter model access, controlled Python tools, function calling, short-term conversation state and safe handling of unsupported requests.

Objectives

- Translate the airline problem into functional and non-functional requirements.
- Build a beginner-friendly Streamlit chat interface.
- Configure an LLM through OpenRouter using a free-model route.
- Create focused tools for flight, booking, baggage and escalation operations.
- Implement function calling and return tool results to the model.
- Maintain short-term conversation state for follow-up questions.
- Test normal, invalid, ambiguous and unsupported requests.
- Document architecture, implementation, test evidence and final outcome.

Project Scope

In Scope	Out of Scope
Synthetic flights, bookings, baggage and support tickets	Real airline systems
Chat UI and conversational state	Real payment/refund processing
Read-only lookup and simulated calculations	Changing real reservations
Mock escalation	Sending real customer tickets
Function calling	Production deployment
Testing and demonstration	Live operational use

7-Session Project Map [Academic Team]

Session	Focus	Deliverable
1	Requirements + wireframe	Problem statement, user stories, UI sketch
2	Chatbot interface	Working Streamlit UI
3	Tools + functions	Mock data + 4–6 tools
4	LLM + function calling	Working agent
5	Stateful support assistant	Memory + error handling
6	Testing + evaluation	10+ test cases + evidence
7	Final demo	Packaged app + presentation

LAB OUTCOME A working browser-based support assistant that can answer airline questions, call approved functions, remember relevant context and explain limitations.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

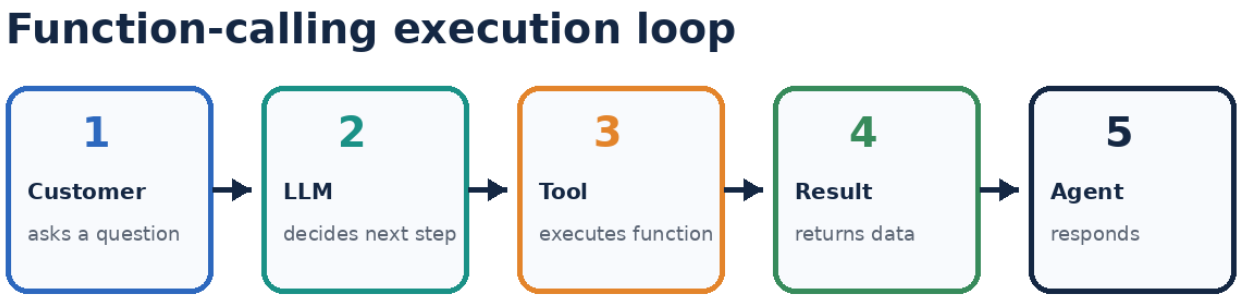
2. Theory

2.1 What is SkyAssist?

SkyAssist is a single-model conversational application. The LLM interprets the customer's request, decides whether an available function is needed and uses the returned result to formulate the answer. The application—not the model—controls and executes the functions.

2.2 Function Calling

OpenRouter's current tool-calling documentation describes a three-part pattern: send the model a request with tool definitions; execute the requested tool locally; then send the tool result back so the model can produce the final response.



KEY PRINCIPLE The model proposes a tool call; your Python application validates and executes the approved function.

2.3 Why Synthetic Airline Functions?

Synthetic functions let students learn agent engineering without access to private airline APIs or exposure of personal information. Each function behaves like a miniature backend service.

2.4 Stateful Conversation

State allows follow-up questions such as “Is it delayed?” after the customer has already said “My flight is SA102.” In the recommended **LangChain** implementation, short-term memory can be represented as thread-level agent state with a checkpointer. Streamlit session state can also retain the visible UI history.

2.5 Why Single Model?

- Keeps the architecture aligned with the Semester 1 requirement for the course ADAI Agents.
- Makes tool calling and memory the focus rather than multi-agent orchestration.
- Is easier for beginners to debug and evaluate.
- Recommended tool is StreamLit, CrewAI can be used as an alternative framework, but multiple agents are not required.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

3. Schematic Diagram

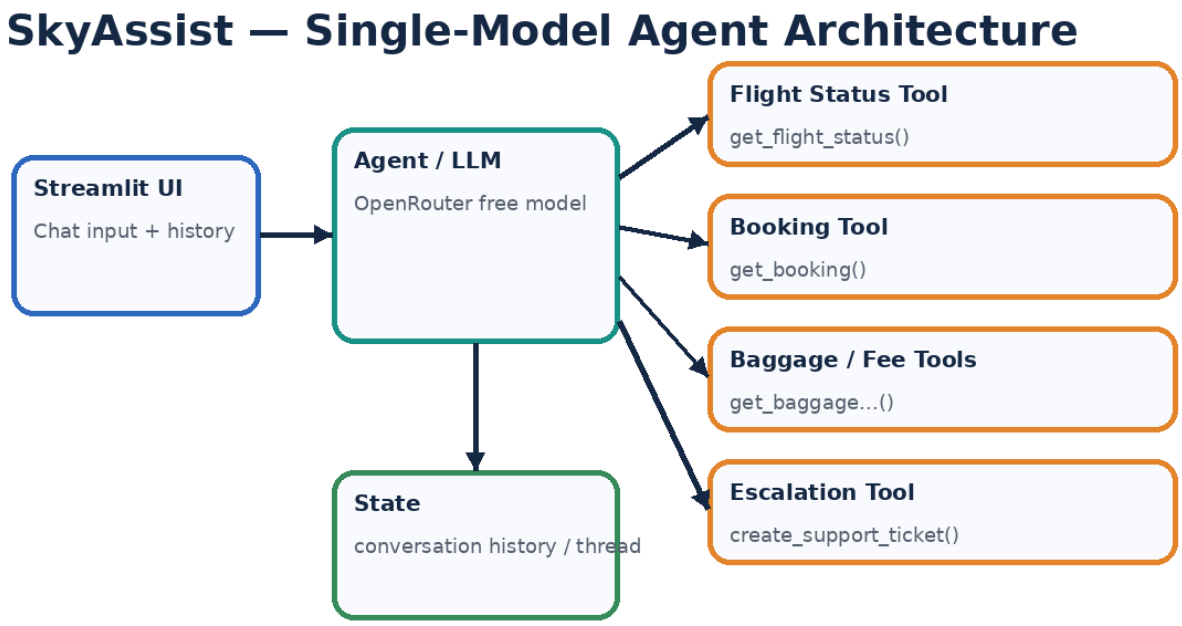


Figure: Recommended SkyAssist architecture

Layer	Responsibility
Streamlit UI	Collect input, display conversation and provide reset/configuration controls.
Agent / LLM	Interpret request, select tools and formulate the response.
Tools	Execute focused Python functions against synthetic airline data.
State	Maintain relevant context and isolate conversations.
Mock data	Provide deterministic flight, booking and policy records.
Evaluation	Check tool selection, arguments, results, response quality and safety.

OPTIONAL CREWAI TRACK Implement the same single-agent design with CrewAI tools if required. Do not introduce multiple agents unless it is an extension task.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

4. Pre-Lab Setup

4.1 Recommended Environment

Component	Choice	Purpose
OS	Windows / macOS / Linux	Development machine
Python	3.11+ recommended	Runtime
Editor	**VS Code [Recommended]** / PyCharm / Jupyter	Development
UI	Streamlit	Browser chat interface
LLM access	OpenRouter	Model gateway
Model route	openrouter/free	Free-model routing
Agent framework	LangChain / LangGraph	Agent + tools + state
Data	Python dicts / JSON	Synthetic backend
Version control	Git	Project history

4.2 Create Project Folder

```
skyassist/
├── app.py
├── tools.py
├── data.py
├── agent.py
├── requirements.txt
├── README.md
├── tests/
└── .streamlit/
    └── secrets.toml
```

4.3 Create Virtual Environment

```
python -m venv .venv

# Windows PowerShell
.\.venv\Scripts\Activate.ps1

# macOS / Linux
source .venv/bin/activate
```

4.4 Install Packages [Student can also use uv package manager]

```
pip install -U streamlit openai python-dotenv langchain langgraph
```

CHECKPOINT Run `python --version` and `streamlit --version`. Record versions in README.

4.5 Configure OpenRouter

OpenRouter provides a unified API and an OpenAI-compatible interface. Its current `openrouter/free` route selects among available free models and filters for supported capabilities such as tool calling. Free-model availability and limits can change, so verify them immediately before class.

```
# .streamlit/secrets.toml
OPENROUTER_API_KEY = "PASTE_YOUR_KEY_HERE"
```

SECURITY Note: Never commit secrets.toml, .env files or API keys to Git.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

4.6 First LLM Connectivity Test

```
from openai import OpenAI
import streamlit as st

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=st.secrets["OPENROUTER_API_KEY"],
)

response = client.chat.completions.create(
    model="openrouter/free",
    messages=[{"role": "user", "content": "Say hello to SkyAssist."}],
)

print(response.choices[0].message.content)
```

EXPECTED A model-generated greeting should be returned. Exact wording is variable.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

5. Project Steps

Task 1 — Requirements and Wireframe

Session 1 • Target: 3–4 hours

Step 1.1 — Define the user

- Primary user: airline passenger/customer.
- Secondary user: student developer/support administrator.
- Expectation: fast, clear answers without understanding backend functions.

Step 1.2 — User Stories

ID	User Story	Acceptance Check
US01	Check a flight status.	Flight tool is used for a known flight.
US02	Look up a booking.	Booking tool returns the synthetic record.
US03	Ask a follow-up question.	Agent uses conversation context.
US04	Request unsupported action.	Agent explains limitation; no false claim.
US05	Inspect tool usage.	Developer can observe the selected function.

Step 1.3 — Wireframe

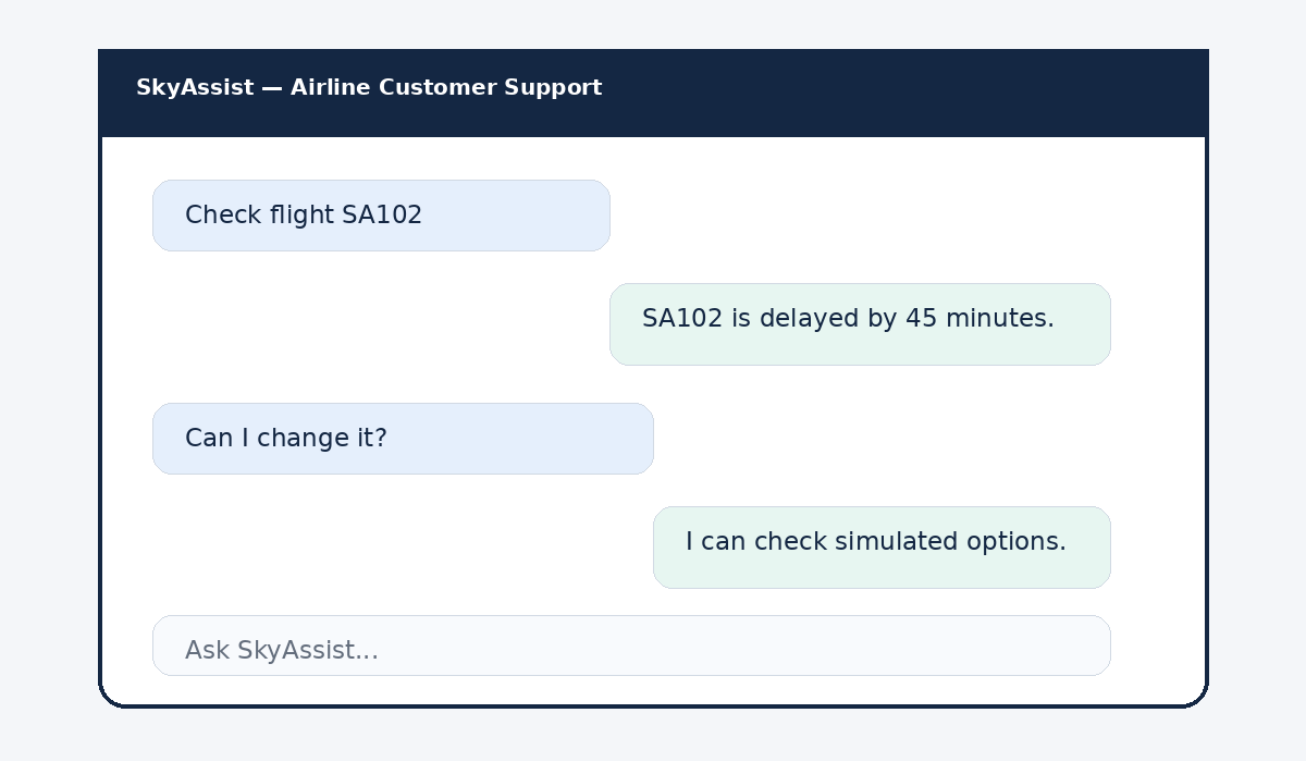


Figure: Beginner-friendly Streamlit wireframe

DELIVERABLE Submit the wireframe and one-page requirements sheet before coding the agent.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

Task 2 — Build the Chatbot Interface

Session 2 • Target: 3–4 hours

```
import streamlit as st

st.set_page_config(page_title="SkyAssist", page_icon="(flight icon)")
st.title("SkyAssist")
st.caption("Airline customer support assistant — academic prototype")

if "messages" not in st.session_state:
    st.session_state.messages = []

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

prompt = st.chat_input("How can I help with your flight?")

if prompt:
    st.session_state.messages.append(
        {"role": "user", "content": prompt}
    )

    with st.chat_message("user"):
        st.markdown(prompt)

    with st.chat_message("assistant"):
        st.markdown("Demo response — agent will be connected next.")
```

Streamlit's current chat API provides `st.chat_message` for messages and `st.chat_input` for customer input. Its official tutorial also demonstrates `st.session_state` for chat history.

CHECKPOINT Run `streamlit run app.py`. Enter three messages and confirm they remain visible after reruns.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

5. Project Steps — Tools

Task 3 — Build the Mock Airline Backend

Session 3 • Target: 4 hours

Step 3.1 — Create synthetic data

```
FLIGHTS = {
    "SA102": {"status": "Delayed", "delay_minutes": 45,
              "gate": "B12", "destination": "Mumbai"},
    "SA205": {"status": "On Time", "delay_minutes": 0,
              "gate": "A08", "destination": "Delhi"},
}

BOOKINGS = {
    "SKY1001": {"passenger": "Demo User", "flight": "SA102",
                "status": "Confirmed", "baggage": 1},
    "SKY1002": {"passenger": "Demo User 2", "flight": "SA205",
                "status": "Confirmed", "baggage": 2},
}
```

WHY THIS MATTERS A real agent would call enterprise systems. Here, deterministic mock data keeps the project safe and beginner-friendly.

Step 3.2 — Create the first function

```
def get_flight_status(flight_number: str) -> dict:
    flight = FLIGHTS.get(flight_number.upper())
    if not flight:
        return {"found": False,
                "message": f"Flight {flight_number} was not found."}
    return {"found": True,
            "flight_number": flight_number.upper(),
            **flight}
```

Step 3.3 — Add the remaining functions

Function	Input	Purpose
get_booking	booking_id	Return synthetic booking record
get_baggage_allowance	booking_id	Return included baggage
calculate_extra_bag_fee	bags	Return simulated fee
check_flight_change_options	booking_id	Return simulated alternatives
create_support_ticket	summary, priority	Create mock escalation record

Step 3.4 — Test tools directly

```
print(get_flight_status("SA102"))
print(get_flight_status("XX999"))
print(get_booking("SKY1001"))
```

SUCCESS CRITERIA Every tool works correctly from Python before the LLM is allowed to select it.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

5. Project Steps — LLM and Function Calling

Task 4 — Connect the LLM and Tools

Session 4 • Target: 4 hours

Step 4.1 — Create a support system prompt

```
SYSTEM_PROMPT = """
You are SkyAssist, an airline customer support assistant.

Rules:
1. Be concise, professional and helpful.
2. Use tools when structured airline data is required.
3. Never invent flight status, booking status or fees.
4. Ask for missing identifiers.
5. Never claim a real booking change, refund or payment occurred.
6. Escalate unsupported or sensitive requests.
7. Use relevant conversation context for follow-up questions.
"""
```

Step 4.2 — Define a tool schema

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_flight_status",
            "description": "Get simulated status information for an airline flight.",
            "parameters": {
                "type": "object",
                "properties": {
                    "flight_number": {
                        "type": "string",
                        "description": "Flight number such as SA102"
                    }
                },
                "required": ["flight_number"]
            }
        }
    }
]
```

Step 4.3 — Map names to functions

```
TOOL_MAP = {
    "get_flight_status": get_flight_status,
    "get_booking": get_booking,
    "get_baggage_allowance": get_baggage_allowance,
    "calculate_extra_bag_fee": calculate_extra_bag_fee,
    "check_flight_change_options": check_flight_change_options,
    "create_support_ticket": create_support_ticket,
}
```

BEGINNER RULE Keep each tool's name, description, schema and Python function aligned. Mismatches are a common source of errors.

Step 4.4 — Start the tool loop

```
response = client.chat.completions.create(
    model="openrouter/free",
    messages=messages,
    tools=tools,
)
```

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

```
assistant_message = response.choices[0].message

if assistant_message.tool_calls:
    # Parse the requested function and JSON arguments.
    # Execute only a function in TOOL_MAP.
    # Append the tool result to the conversation.
    # Send the updated messages back to the model.
    pass
else:
    final_answer = assistant_message.content
```

During development, inspect the returned response object because SDK/model behavior can vary.
Do not fabricate a tool result; execute the mapped function.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

5. Project Steps — Stateful Support Assistant

Task 5 — Add Agent Runtime and Memory

Session 5 • Target: 4 hours

For the recommended implementation, use the current LangChain `create_agent` API with a checkpointer for thread-level short-term memory.

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

agent = create_agent(
    model=model,
    tools=[
        get_flight_status,
        get_booking,
        get_baggage_allowance,
        calculate_extra_bag_fee,
        check_flight_change_options,
        create_support_ticket,
    ],
    system_prompt=SYSTEM_PROMPT,
    checkpointer=checkpointer,
)
```

Step 5.1 — Continue a conversation with a thread ID

```
config = {"configurable": {"thread_id": "demo-user-001"}}

agent.invoke(
    {"messages": [{"role": "user",
                   "content": "My booking is SKY1001."}]},
    config,
)

agent.invoke(
    {"messages": [{"role": "user",
                   "content": "Is my flight delayed?"}]},
    config,
)
```

EXPECTED The second turn can use relevant state from the first turn. Exact wording is model-dependent.

Step 5.2 — Reset and isolate conversations

- Use a new thread ID for a fresh conversation.
- Provide a visible Reset Chat control.
- Never reuse another user's thread ID.
- Do not persist real personal information in the academic prototype.

Step 5.3 — Optional CrewAI Track

CrewAI supports agents with tools, memory and workflows. If your course requires CrewAI, implement the same six Python tools with one CrewAI agent. Keep the business scope identical so students can compare frameworks without changing the project requirements.

FRAMEWORK CHOICE Recommended beginner path: LangChain/LangGraph because it directly supports the course's current agent + memory concepts. CrewAI is an optional alternative.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

5. Project Steps — Integration, Testing and Demo

Task 6 — Integrate UI + Agent + Tools + State

Session 5 • Complete the end-to-end application

1. Start Streamlit and initialize the agent/checkpointer.
2. Render existing conversation history.
3. Accept customer input with `st.chat_input`.
4. Append the customer message to visible chat.
5. Invoke the agent using the current thread ID.
6. Render the final assistant response.
7. During development, inspect/log tool calls and errors.
8. Provide Reset Chat.

Error Handling Matrix

Situation	Expected behavior
Unknown flight	Ask customer to verify the flight number.
Unknown booking	Ask customer to verify the booking ID.
Tool exception	Safe support message; technical detail stays in logs.
Missing identifier	Ask one targeted clarification.
Unsupported transaction	Explain limitation and offer escalation.
Repeated failure	Offer simulated human escalation.

Task 7 — Test the Agent

Session 6 • Target: 3–4 hours

ID	Customer input	Expected behavior	Pass
TC01	Economy baggage allowance?	Policy answer	☐
TC02	Check flight SA102.	Flight tool	☐
TC03	Find booking SKY1001.	Booking tool	☐
TC04	My booking is SKY1001. Is it delayed?	Context + flight tool	☐
TC05	Cost for one extra bag?	Fee tool	☐
TC06	Can I change SKY1001?	Change-options tool	☐
TC07	Check XX999.	Graceful not-found	☐
TC08	Find ABC999.	Graceful not-found	☐
TC09	Refund my card now.	Limit / escalate	☐
TC10	Start a new conversation.	Old state not reused	☐

What to Record

- Input
- Selected tool
- Arguments
- Tool result
- Final response
- Pass/fail
- Observed issue
- Screenshot/trace

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

Task 8 — Final Demo

Session 7 • Target: 3–4 hours

1. Introduce the business problem.
2. Show the Streamlit interface.
3. Demonstrate a normal support question.
4. Demonstrate a tool-backed lookup such as SA102.
5. Demonstrate a stateful follow-up.
6. Demonstrate one invalid and one unsupported request.
7. Show test evidence and architecture.
8. Explain limitations and future enhancements.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

6. Quick Reference

Tool Catalog

Tool	Input	Example	Safe scope
get_flight_status	flight_number	SA102	Read synthetic flight
get_booking	booking_id	SKY1001	Read synthetic booking
get_baggage_allowance	booking_id	SKY1001	Read allowance
calculate_extra_bag_fee	bags	1	Simulated calculation
check_flight_change_options	booking_id	SKY1001	Simulated alternatives
create_support_ticket	summary, priority	Delay complaint	Mock escalation

Commands

```
# Activate
.\.venv\Scripts\Activate.ps1

# Install
pip install -r requirements.txt

# Run
streamlit run app.py

# Optional: freeze environment
pip freeze > requirements.txt
```

Suggested requirements.txt

```
streamlit
openai
python-dotenv
langchain
langgraph
```

Troubleshooting

Problem	Likely cause	Action
401 / unauthorized	Missing/invalid key	Check Streamlit secrets and key status.
Model unavailable	Free route changed	Check OpenRouter current free models.
No tool call	Tool support/schema issue	Verify tool support and descriptions.
Wrong tool	Overlapping descriptions	Make tool purposes distinct.
Memory not working	Thread ID changes	Reuse same thread ID.
State resets	Not stored correctly	Use st.session_state/checkpointer.
Import error	Package mismatch	Record versions and install compatible versions.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

7. Output

Expected Final Application

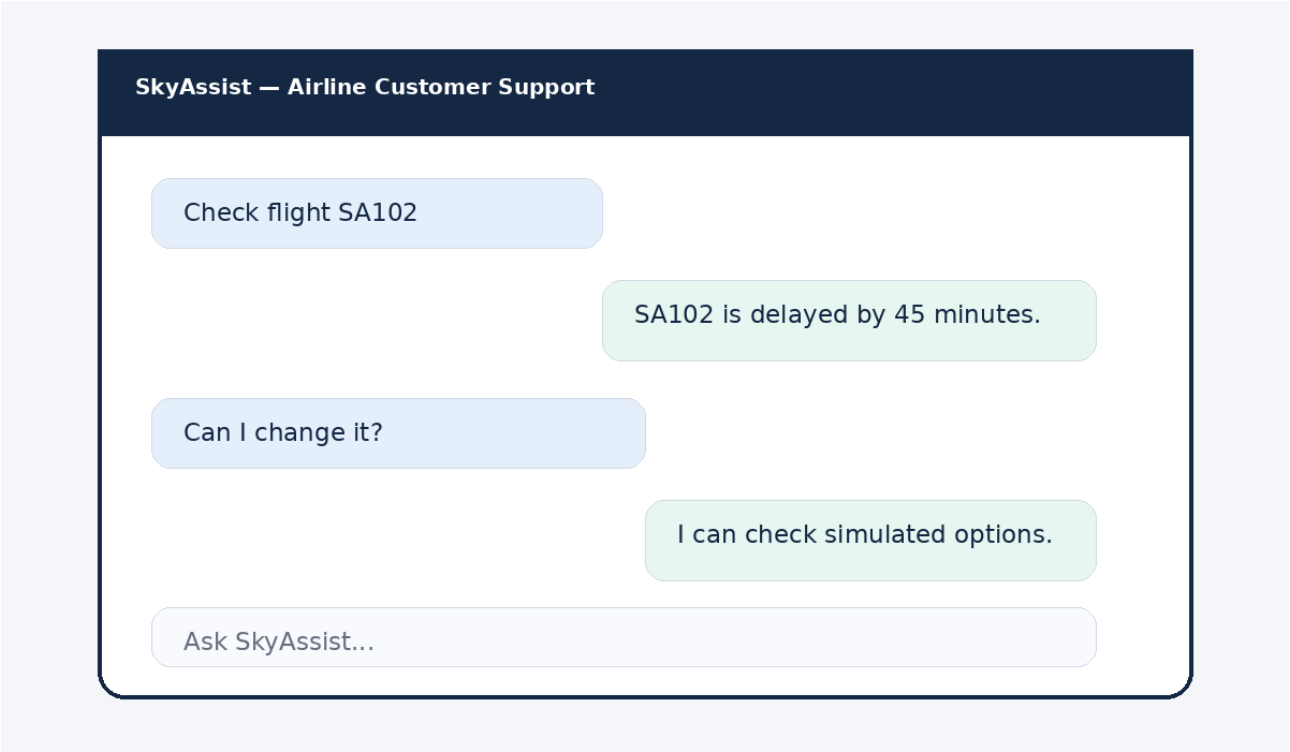


Figure: Illustrative target interface — student designs may differ.

Minimum Demonstrable Outcome

- Browser-based Streamlit chatbot runs.
- Customer and assistant turns are visible.
- LLM is reached through OpenRouter.
- At least four airline functions are available.
- At least three scenarios demonstrate successful function calling.
- At least one follow-up demonstrates state.
- At least two failure cases are handled gracefully.
- README explains setup and execution.

Evidence Checklist

- Screenshot of working UI
- Screenshot/trace of a successful tool call
- Screenshot of stateful follow-up
- Screenshot of graceful error
- Test-case table with pass/fail
- Architecture diagram
- README + requirements.txt
- Final presentation/demo

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

8. Evaluation Criteria

Sr.	Criteria	Marks (%)
1	Understanding of aim, requirements and architecture	10
2	Project environment, Streamlit UI and reproducible setup	15
3	LLM integration through OpenRouter	15
4	Function/tool design and working function calling	20
5	Stateful conversation and reset/isolation	10
6	Error handling, safety and unsupported-request handling	10
7	Test cases, evidence and evaluation quality	10
8	Documentation, code quality and packaging	5
9	Final presentation and demonstration	5

Evaluation Guidance

- Assess both tool selection and actual Python execution.
- Do not award full marks for a visually attractive chatbot without working function calls.
- Students should explain limitations instead of hiding failed cases.
- Grade LLM behavior against requirements rather than exact wording.
- API-key exposure or use of real customer data is a serious project-quality issue.

SUCCESS CRITERIA The student can run the application, explain every major component, demonstrate tool use and a memory-based follow-up, and defend the test results.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

9. Reference Links

Use these official resources for setup and current API behavior. Model availability, quotas and framework APIs can change.

Resource	Category	Purpose	Link
OpenRouter Quickstart	LLM/API	API setup and OpenAI-compatible access	https://openrouter.ai/docs/quickstart
OpenRouter Free Models Router	LLM/API	Current free routing and capability filtering	https://openrouter.ai/openrouter/free
OpenRouter Tool Calling	Function calling	Tool schema and execution loop	https://openrouter.ai/docs/guides/features/tool-calling
Streamlit Chat Elements	UI	st.chat_message / st.chat_input	https://docs.streamlit.io/develop/api-reference/chat
Streamlit Chat Tutorial	UI	Beginner conversational app	https://docs.streamlit.io/develop/tutorials/chat-and-llm-apps/build-conversational-apps
LangChain Agents	Agent framework	Current create_agent and tools	https://docs.langchain.com/oss/python/langchain/agents
LangChain Short-Term Memory	Memory	Thread-level state/checkpointer	https://docs.langchain.com/oss/python/langchain/short-term-memory
LangChain Tools	Tools	Python tool definitions	https://docs.langchain.com/oss/python/langchain/tools
CrewAI Documentation	Alternative framework	Agents, tools, crews and flows	https://docs.crewai.com/

Further Study

- Verify current OpenRouter free-model availability and tool-calling support before selecting a fixed model.
- Study Streamlit chat elements and session state.
- Study LangChain `create_agent`, tools and short-term memory.
- Compare the same project implemented with CrewAI as an optional extension.

STRUCTURE NOTE
The SkyAssist guide follows that structure and uses reference as Overview → Theory → Schematic Diagram → Pre-Lab Setup → Project Steps → Quick Reference → Output → Evaluation Criteria → Reference Links.

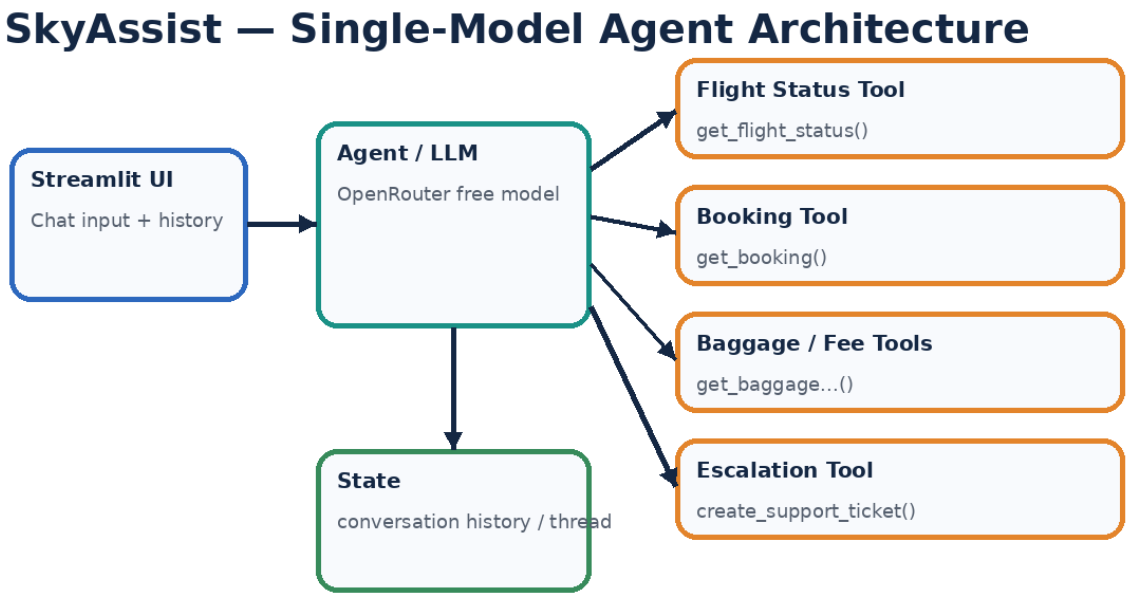
Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®

FINAL PROJECT OUTCOME

SkyAssist

A beginner-built, single-model airline customer-support agent with UI, controlled tools, function calling and conversation state.



TAKEAWAY
Success means students can explain not only what the agent answered, but how the request moved through UI → model → tool → state → final response.

Disclaimer: This project is intended for educational purpose. All names, organizations, and scenarios used within are entirely fictitious and are not intended to represent any real-world entity.

Jetking®